G Growlancer

Growlancer Security Audit Prompts

5 codebase-specific security checks — customized for Growlancer's exact stack: Supabase, Razorpay + PayPal, escrow/wallets, and Vike/Express.

Unlike generic security checklists, every prompt below references your actual files, Supabase tables, RPCs, and Edge Functions — pulled directly from your uploaded codebase (Growlancer-main.zip). Paste each one into Emergent (or Claude Code / Cursor) in order.

Stack detected	React 19 + Vike (SSR) + Express + Supabase
Payments	Razorpay (India) + PayPal (international), both with webhooks
Money system	Escrow RPCs + Wallets + Withdrawal/Payout
Auth	Supabase Auth + custom MFA/2FA with recovery codes
AI features	Gemini-powered ai-assistant, ai-matching, ai-ticket-responder
Admin	admin-data, admin-signup Edge Functions
Compliance	request-deletion / process-deletion (GDPR-style account erasure)

Based on the Emergent security-prompt methodology by Mayank Shah (@mayankshah_ai) — adapted line-by-line for Growlancer's codebase.

G Growlancer

How to use this guide

Paste the five prompts into Emergent (or whatever AI coding tool has access to your Growlancer repo) one at a time, in order. Each prompt ends by asking it to report exactly what it found and changed — read that summary, don't just paste and move on.

Run in this order

1. **Secret Leak Prevention**
   Supabase keys, Razorpay/PayPal keys, Gemini key, VITE_ exposure

2. **Personal Data Flow Audit**
   KYC data, wallet/payout details, AI data sent to Gemini, account deletion

3. **Pre-Deploy Production Audit**
   Express server headers, rate limiting, CORS on Edge Functions

4. **Deep Security Audit**
   Escrow/wallet RPCs, Razorpay+PayPal webhook verification, MFA storage

5. **Attacker's Perspective Review**
   IDOR on contracts/wallets, admin bypass, referral/trial abuse

One honest note

These prompts catch the mistakes that cause most real-world breaches in vibe-coded apps — but no AI audit replaces professional security testing. Growlancer moves real money through Razorpay and PayPal escrow, so before a public launch, pair this with a human security review, especially for the wallet/withdrawal RPCs.

G Growlancer

CHECK 1 OF 5

Secret Leak Prevention

Based on: Gitleaks — customized for Growlancer's Supabase + Razorpay + PayPal + Gemini setup

Growlancer uses Vite/Vike, so any env var prefixed `VITE_` is bundled straight into the client JS. This check confirms nothing sensitive uses that prefix, and that your two payment gateways plus AI key are properly isolated to server-side Edge Functions.

PASTE THIS INTO EMERGENT

```
Before deploying Growlancer, do a full secret safety pass across the entire codebase (src/, supabase/functions/, server.js). Here is exactly what to check and fix:

1. VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY in src/lib/supabase.ts are meant to be client-side, but ONLY if Row Level Security is enabled on every table used by the anon/authenticated roles. Confirm RLS is ON for every table referenced in supabase/migrations/ — list any table where it is missing or was disabled by a later migration.

2. The Supabase SERVICE_ROLE key must never appear in src/ or in any client-bundled file. It should only be referenced inside supabase/functions/* (Deno Edge Functions: admin-data, admin-signup, withdrawal, delete-account, process-deletion). Search the whole src/ directory for any string containing 'service_role' or a raw JWT starting with 'eyJ' and flag it immediately.

3. Check RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_ACCOUNT_NUMBER (used in src/lib/razorpay.ts and supabase/functions/razorpay, razorpay-payout-webhook) — only the key ID may ever reach the client; the secret and account number must stay inside Edge Functions only.

4. Check PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_WEBHOOK_ID (src/lib/paypal.ts, supabase/functions/paypal, paypal-webhook) — same rule: secret and webhook ID server-side only.

5. Check GEMINI_API_KEY (used by supabase/functions/ai-assistant, ai-matching, ai-ticket-responder) — must only be read inside those Edge Functions, never in src/lib/aiMatching.ts on the client side.

6. Confirm .env is in .gitignore and that .env.example (already present) contains only placeholders — verify no real key was ever committed by checking git log -p for .env.

7. Check every console.log across supabase/functions/*/index.ts — none should print the Razorpay secret, PayPal secret, Gemini key, service role key, or a full Supabase JWT.

Show me a summary of every secret you found, exactly which file/line it was in, and what you
```

```
moved it to.
```

G Growlancer

CHECK 2 OF 5

Personal Data Flow Audit

Based on: Bearer — customized for Growlancer's KYC verification, payouts, and Gemini AI integration

Growlancer collects more than average — identity/credential verification for KYC, payout bank/UPI details, portfolio files, and it sends conversation data to Google's Gemini API for AI matching. This check traces all of it.

PASTE THIS INTO EMERGENT

```
Do a full audit of how user personal data moves through Growlancer. I need to know exactly where sensitive data enters, travels, and ends up.

1. Map these specific collection points and trace where each one goes: identityVerification.ts and credentialVerification.ts (KYC-style documents), clientPaymentMethods.ts and withdrawal.ts (bank/UPI/payout details), avatarUpload.ts / portfolioImageUpload.ts / fileUpload.ts (uploaded files), messages.ts (direct messages between client and freelancer), reviews.ts.

2. Clean every console.log/logger in supabase/functions/*/index.ts and src/lib/*.ts. None should print emails, phone numbers, payout account numbers, Razorpay/PayPal order details, or Supabase auth tokens.

3. Audit the AI data flow specifically: supabase/functions/ai-assistant, ai-matching, and ai-ticket-responder all call Google's Gemini API via GEMINI_API_KEY. List exactly what user fields (name, email, project details, message content) are sent in each request payload to Gemini, and strip any field the AI feature doesn't actually need.

4. MFA and recovery codes: check the mfa_and_recovery_codes migration and twofa-management Edge Function — recovery codes must be hashed before storage (never stored plaintext), and the TOTP secret must not be returned in any API response after initial setup.

5. Wallet and payout data: check wallets, payout_methods tables and withdrawal.ts / payout Edge Functions — bank account numbers and UPI IDs must not appear in logs, and admin-data must not expose one user's payout details to another.

6. Cookie/storage audit: confirm no PII (KYC status, payout details, tokens) is written to localStorage — only Supabase's own session storage should hold auth tokens.

7. API response filtering: check admin-data Edge Function specifically — it should only return fields an admin actually needs, never raw payout account numbers or KYC document URLs unless explicitly required.

8. Data deletion: verify request-deletion → process-deletion → the user_deletion_requests migration actually results in personal data being erased or anonymized (not just an account
```

status flag), including wallet history, KYC docs, and messages.

Show me a complete map: what's collected, where it's stored, where it's sent externally (especially to Gemini), and what you fixed.

G Growlancer

CHECK 3 OF 5

Pre-Deploy Production Audit

Based on: ECC Production Audit — customized for Growlancer's Express server + 24 Supabase Edge Functions

Growlancer ships via a custom Express server.js serving a Vike SPA build, plus ~24 separate Supabase Edge Functions. Each surface needs its own pre-deploy pass — headers on the Express server, CORS on every function, and confirmation that the rate_limits table is actually enforced everywhere it should be.

PASTE THIS INTO EMERGENT ■

```
Growlancer is about to be deployed to Vercel. Run through every check below against the real code and fix anything that fails.

1. Environment variables: confirm src/lib/supabase.ts throws a clear error if VITE_SUPABASE_URL or VITE_SUPABASE_ANON_KEY is missing (it currently does — verify this still holds). Do the same check for every supabase/functions/*/index.ts: each must fail loudly if RAZORPAY_KEY_SECRET, PAYPAL_CLIENT_SECRET, or GEMINI_API_KEY is missing, not silently continue.

2. Debug code removal: search src/ and supabase/functions/ for leftover console.log debugging, commented-out blocks, TODO/FIXME comments about incomplete security, and any test-only routes. Confirm import.meta.env.DEV-gated code (like the debug flag in supabase.ts) is actually off in the production build.

3. Error handling in server.js: the current catch-all error handler returns a generic 'Internal server error' — good. Confirm none of the supabase/functions/*/index.ts handlers leak stack traces, SQL error text, or file paths in their JSON error responses to the client.

4. Security headers: server.js currently has no helmet-style headers. Add X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security, and a Content-Security-Policy to every response served from server.js. Also verify the Vercel deployment config sets equivalent headers if server.js is bypassed there.

5. Rate limiting: the rate_limits table (migration 20260610130000) exists — confirm it's actually called (not just defined) inside withdrawal, paypal, razorpay, 2fa-management, and any OTP/login-adjacent Edge Functions. Minimum: 5 attempts/minute on auth-related routes, stricter on withdrawal and 2fa-management.

6. CORS: check the CORS headers set inside every file in supabase/functions/ — none should allow '*' as Access-Control-Allow-Origin for functions that handle payments, wallets, or admin data. Restrict to the production Growlancer domain.

7. Database security: confirm the Supabase Postgres connection enforces TLS (default on Supabase, but verify no direct connection string is hardcoded anywhere outside env vars), and that supabase/.temp is in .gitignore.
```

```
List every check, whether it passed or failed, and exactly what you changed.
```

G Growlancer

CHECK 4 OF 5

Deep Security Audit for Complex Logic

Based on: Trail of Bits Skills — customized for Growlancer's escrow RPCs, dual payment gateways, and MFA

This is the important one. Growlancer has two live payment gateways, an escrow system, a wallet/payout ledger, and custom MFA — exactly the class of app this check exists for. Your escrow RPCs already enforce `auth.uid() = p_client_id` — this check verifies that holds everywhere and that both webhook integrations are locked down.

PASTE THIS INTO EMERGENT ■

```
Growlancer has Razorpay + PayPal payments, an escrow/wallet/payout system built on Supabase
RPCs, and custom MFA on top of Supabase Auth. Do a deep security audit on these critical
paths:

AUTHENTICATION & AUTHORIZATION
Check every protected route and every Edge Function in supabase/functions/ for proper auth
verification (JWT check before any DB write). Specifically verify there's no IDOR in
contractMilestones.ts, workflowService.ts, disputeService.ts, portfolio.ts, and messages.ts
— none of these should accept a contract_id, milestone_id, or conversation_id from the client
and return/modify it without confirming the caller is a party to it. Review the
twofa-management function and the mfa_and_recovery_codes migration — recovery codes must be
single-use and hashed, and 2FA cannot be disabled without re-authentication.

ESCROW & WALLET RPCs
The escrow_rpcs migration (20260610110000) already checks 'p_client_id <> auth.uid()' inside
SECURITY DEFINER functions — confirm every RPC in escrow_rpcs.sql, wallet_rpcs.sql
(20260525160000), and any function touching wallets has the equivalent ownership check, with
no path where a caller can pass someone else's user_id and move funds. Also re-check
anon/authenticated grants against
20260519095656_revoke_anon_execute_on_sensitive_definer_rpcs.sql — confirm no RPC added
after that migration was granted back to anon by mistake.

PAYMENT LOGIC — RAZORPAY + PAYPAL
Never trust a client-submitted amount. Confirm src/lib/razorpay.ts and src/lib/paypal.ts
always have the Edge Function (supabase/functions/razorpay, supabase/functions/paypal)
independently recompute the amount/currency server-side before creating an order — an
attacker should not be able to modify the contract/service price in the request body. Verify
supabase/functions/paypal-webhook and razorpay-payout-webhook both validate the provider's
webhook signature before trusting any payload, and that payment/payout status is only marked
complete after that verification passes, never from a client-side callback alone.

INPUT HANDLING
Check every Edge Function for parameterized Supabase queries (no raw string-concatenated
SQL). Check messages.ts, reviews.ts, and portfolio.ts for XSS — is any user-entered text
```

(bio, review, message, portfolio description) rendered without sanitization anywhere in src/components? Check avatar-upload and file-upload Edge Functions — file type and size must be validated server-side (not just in the browser), and uploaded files must not be served with executable permissions from the same domain.

For every issue found, show me: what the vulnerability is, exactly where it is in the code, how an attacker would exploit it, and the precise fix.

G Growlancer

CHECK 5 OF 5

Attacker's Perspective Review

Based on: ECC Security Review — customized for Growlancer's referral system, admin functions, and AI cost-abuse surface

The final pass — think like someone trying to break Growlancer specifically. This covers the referral system, subscription trial logic, and admin functions that exist in your codebase, plus the standard IDOR/privilege-escalation checks every marketplace needs.

PASTE THIS INTO EMERGENT ■

```
Think like an attacker trying to break Growlancer. Check these specific attack paths against the real code:

1. Data access via ID manipulation. On every endpoint that takes a contract_id, milestone_id, wallet_id, conversation_id, or dispute_id (contractMilestones.ts, workflowService.ts, disputeService.ts, messages.ts, withdrawal.ts) — can I substitute another user's ID and read or modify their data? Ownership must be checked server-side, not just filtered in the UI.

2. Login/session bypass. Check if any supabase/functions/*/index.ts skips JWT verification. Check that expired or malformed tokens are rejected everywhere, not just on some routes. Check admin-signup — confirm it can't be reached to self-provision an admin account without an existing admin's authorization.

3. Privilege escalation. admin-data and admin-signup are the two admin-facing functions — confirm role is checked server-side inside the function itself (via a DB lookup or claim on the JWT), not just by a client-side route guard in src/app or src/pages. A regular authenticated user should get a 403 calling these directly.

4. Feature abuse — check rate limits actually apply to: signup, the OTP/email-confirmation flow, messaging (spam between users), file uploads (storage fill via avatar-upload/file-upload), and specifically the AI Edge Functions (ai-assistant, ai-matching, ai-ticket-responder) since every call costs a Gemini API credit — an unthrottled loop there is a direct cost-abuse vector.

5. Referral and subscription abuse. Check the logic behind 20260620000000_fix_referral_system.sql — can a user refer themselves with a second account to farm rewards? Check 20260616_update_trial_days_and_plans.sql and subscription-billing-cron — can a user delete and recreate an account (or use request-deletion then re-signup) to restart a free trial indefinitely?

6. Content injection. Try script/HTML payloads in every free-text field — bio, review text, portfolio description, message body, support ticket (ai-ticket-responder consumes this) — and confirm none of it executes when rendered back in the UI.
```

```
7. Internal exposure. Check whether any of these are reachable on the deployed Vercel domain: a .git directory, the raw .env file, the supabase/.temp directory, or the /_health endpoint leaking anything beyond a plain status. Since Supabase auto-exposes tables via PostgREST, also confirm RLS is enabled on every table — not just the sensitive-looking ones — since PostgREST will otherwise serve raw rows to anyone with the anon key.

8. Business logic manipulation. On escrow contract creation — can the client submit a negative or zero amount? Can a discount/promo code be applied more than once on the same contract? Can a withdrawal be triggered for more than the actual wallet balance shown in the wallets table?

For every vulnerability found: explain what an attacker would do, how much damage they could cause, and fix it immediately. Fund/data theft first, abuse and logic flaws second.
```

You're done — almost

Re-run Prompt 5 after any major feature update — new code means new attack surface, especially around the escrow/wallet RPCs and the two payment webhooks. Growlancer handles real money through Razorpay and PayPal escrow, so pair these five prompts with a human security review before a public launch.

Prompt methodology based on Emergent Security Prompts by Mayank Shah (Instagram/YouTube @mayankshah_ai). Customized for Growlancer's codebase.